

Visibility-Aware Stealth Path Planning with Monte Carlo Graph Search

Allison Meikle

COMP 400 – Honours Research Project
McGill University

Supervised by Professor Clark Verbrugge

December 2025

Introduction

Stealth games form a distinct and influential genre in interactive entertainment, exemplified by titles such as *Metal Gear Solid*, *Splinter Cell*, *Hitman*, and *Dishonored*. In these games, the core challenge is not simply to reach a destination, but to do so while avoiding detection by enemy guards, cameras, or other sensing mechanisms. Player success depends on careful movement through an environment where visibility is constrained by geometry, guard patrols, and line-of-sight, and where small positional decisions can have significant consequences for long-term survival.

From a computational perspective, stealth games pose a challenging planning problem. Guards typically follow deterministic patrol routes and possess limited fields of view, causing the set of visible and hidden regions of the map to change over time. As a result, the environment can be viewed as a continuously evolving visibility landscape, where areas of concealment appear, disappear, and fragment as guards move. The player must constantly adapt their position in response to these changes, balancing short-term safety against future opportunities to remain hidden.

Most prior work on path planning in games and robotics assumes a goal-directed formulation: an agent is given a fixed start state and a predefined destination, and the task is to compute a path that optimizes some criterion such as distance, cost, or exposure. In stealth contexts, this often translates to finding a safe or minimally visible route from point A to point B. While effective for many scenarios, this formulation does not capture an important class of stealth gameplay situations in which no explicit destination exists.

This work considers a fundamentally different problem setting. Rather than planning toward a fixed goal, the agent is tasked with surviving for as long as possible in the presence of moving observers. There is no terminal destination and no requirement to make progress toward a target location. Instead, the objective is to select a sequence of positions over time that maximizes the total duration the agent remains hidden. The search space is therefore defined primarily by time rather than by location, and success is measured by sustained concealment rather than by goal achievement.

This survival-oriented formulation introduces several challenges. The agent must reason over a continuous spatial environment while accounting for dynamically changing visibility constraints induced by guard motion. Locally safe decisions may lead to unavoidable exposure in the future, while temporarily risky movements may be necessary to reach more robust hiding regions later on. Effective planning, therefore, requires long-horizon reasoning, the ability to evaluate future visibility outcomes, and mechanisms for balancing immediate concealment against future safety.

To address this problem, we propose a visibility-aware planning approach based on Monte Carlo Graph Search (MCGS). Our method operates in continuous two-dimensional polygonal environments with deterministic guard patrols, full 360° fields of view, and unbounded visibility range. By explicitly modelling time-varying visibility, extracting geometric summaries of concealment structure, and integrating these into a graph-based Monte Carlo search framework, the planner can reason about both local hiding opportunities and long-term survival strategies without relying on a predefined goal. The remainder of this paper details the formulation, implementation, and evaluation of this approach, demonstrating its effectiveness across a range of stealth scenarios of increasing complexity.

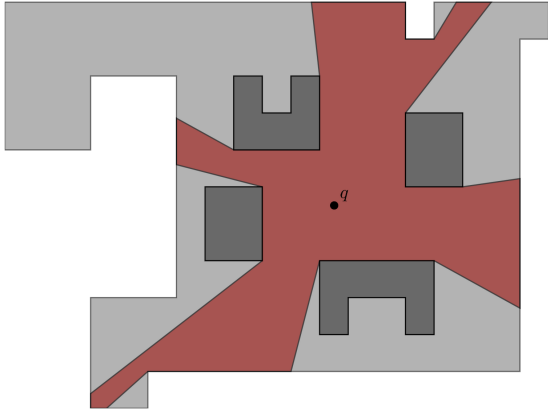
Background

This section provides the necessary background in computational geometry and visibility analysis, introducing the core polygonal representations used to model visibility and concealment, and reviewing related work that applies these concepts to stealth-oriented path planning.

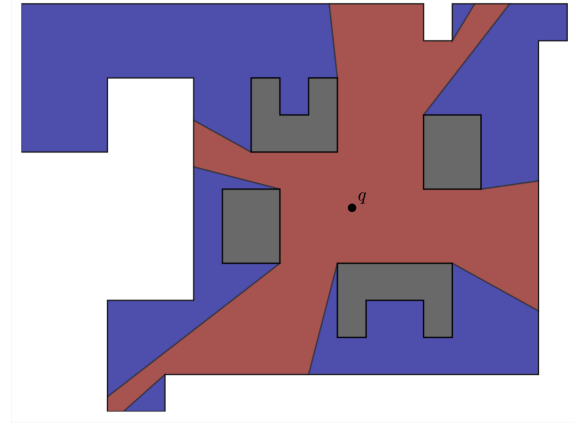
Visibility Polygons

To determine which regions of the environment are visible from a given guard position, this work relies on the concept of *visibility polygons*. Visibility polygon computation is a well-studied problem in computational geometry and is commonly used as a building block for motion planning and visibility reasoning in polygonal environments. Given a planar polygonal environment P , possibly containing polygonal holes, and a query point q in its interior, the visibility polygon is defined as the set of all points $v \in P$ that are directly visible from q . A point v is considered visible if the line segment $\overline{qv}$ lies entirely within P and does not intersect

any boundary or obstacle edge. As illustrated in Figure 1a, the light gray region denotes P , with obstacles shown as darker grey polygons, while the red region corresponds to the visibility polygon of the query point q . Throughout this work, visibility is computed assuming an omnidirectional (360°) field of view (FoV) and is bounded only by the polygonal environment bounds.



(a) The visibility polygon (red) showing all points visible from q within the polygonal environment



(b) The corresponding shadow region (blue), defined as the complement of the visibility polygon within the environment.

Figure 1: Visibility and shadow regions for a guard located at point q with a 360° field of view and unbounded vision range.

A variety of algorithms exist for computing visibility polygons in two-dimensional polygonal environments; in this work, we use a robust angular sweep algorithm implemented in the *VisiLibity* library [Obermeyer and Contributors, 2008]. The algorithm performs a radial sweep centred at the query point q , considering all vertices of the environment boundary and polygonal obstacles. To address geometric degeneracies, small angular perturbations are applied to each vertex direction, generating additional candidate directions for evaluation. Rays are then cast from q in sorted angular order, and for each direction, the nearest intersection with a boundary or obstacle edge is recorded. The set of nearest intersection points obtained during the sweep constitutes the vertex set of the resulting visibility polygon, ordered counter-clockwise by angle.

Shadow Polygons

Given a visibility polygon at a particular timestep, the corresponding *shadow polygon* is defined as the region of the environment that is not visible to the observer. Formally, for a polygonal environment P and a visibility polygon $V \subseteq P$, the shadow region is the set difference $P \setminus V$. This region represents all positions that are safe from detection at that timestep. As shown in Figure 1b, the shadow region (shown in dark blue) may consist of multiple disconnected components due to occlusions caused by obstacles and environment boundaries. Accordingly, the shadow regions in this work are treated as multipolygons, formed by the union of all such distinct shadow polygons.

Polygon Kernels and Inset Polygons

While shadow polygons identify regions that are safe from guard detection at a given timestep, they do not directly indicate which locations within those regions are most protected. To guide movement intelligently, it is therefore useful to reason about the relative depth of points within a shadow region, where deeper points correspond to positions that are farther from visibility boundaries and thus more robust to small changes in observer position.

In computational geometry, this notion of depth and a centrally protected region is formalized by the concept of a polygon *kernel*. A polygon is said to be *star-shaped* if there exists at least one point within the polygon from which every other point in the polygon is directly visible, under the same notion of visibility

defined previously for visibility polygons. The set of all such points is known as the kernel of the polygon [Preparata and Shamos, 1985]. Formally, the kernel consists of all points that have an unobstructed line of sight to every point on the polygon boundary. When a polygon is star-shaped, its kernel identifies the most centrally located region of the polygon.

In the context of stealth and hiding, polygon kernels provide a natural geometric abstraction for identifying well-protected locations within a shadow region. Positions within the kernel are less sensitive to small changes in observer position and are therefore attractive candidates for safe movement and prolonged concealment. However, in realistic stealth environments, as mentioned previously, shadow regions may consist of multiple disconnected components and are often non-convex in shape. As a result, individual shadow polygons are not guaranteed to be star-shaped, and their kernels may be empty, limiting the direct applicability of exact kernel-based reasoning.

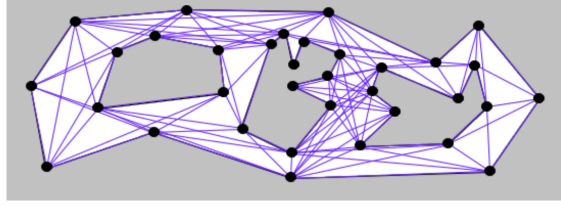
This motivates the use of a kernel approximation approach based on polygon insets, also referred to as buffering or offsetting operations. In this work, polygon insets are implemented using the buffering operation provided by the Shapely geometry library, which performs a uniform inward offset of polygon boundaries [Gillies et al., 2025]. By shrinking a shadow polygon inward, peripheral and narrow regions are eliminated first, leaving behind regions that are increasingly distant from visibility boundaries. Intuitively, points that persist through successive inset operations correspond to locations that are more deeply embedded within the shadow region and can therefore be interpreted as approximate kernel locations. These inset-derived kernel points serve as a practical geometric approximation of polygon kernels, effectively capturing the deepest and safest areas of each shadow region for use in subsequent planning.

Shortest Path Planning in Polygonal Environments

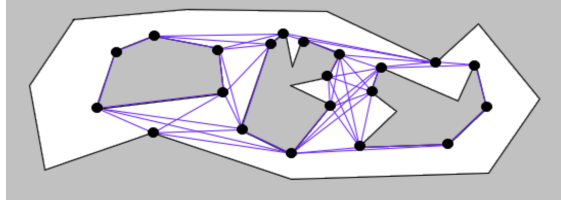
In planar environments with polygonal obstacles, shortest path planning is naturally formulated as a continuous geometric problem: given a start and goal position, compute a shortest path between them through the *walkable area* of the environment. We define the walkable area as the interior of the boundary polygon, excluding the interiors of any obstacle polygons; that is, the subset of the environment in which a character is permitted to move. Importantly, a foundational result in computational geometry shows that this continuous problem can actually be reduced to a discrete graph search problem by exploiting the visibility properties of polygonal vertices. Asano et al. formalize this reduction through the introduction of the *visibility graph* [Asano et al., 1985]. In a visibility graph, vertices correspond to the boundary and obstacle vertices of the environment, as well as the start and goal positions. An edge is constructed between two vertices if they are mutually visible, as defined previously, with the edge weight given by the Euclidean distance between the endpoints. Each edge, therefore, represents a valid straight-line motion through the walkable area. Figure 2a illustrates such a visibility graph constructed over all polygon vertices.

Building on this formulation, the *ExtremityPathFinder* library exploits additional geometric structure to reduce the size of the search graph while preserving its ability to represent all shortest paths [Schneider, 2020]. Rather than including all boundary and obstacle vertices, it restricts the graph to a subset known as *extremities*. An extremity is defined as a polygon vertex whose interior angle, measured with respect to the walkable area, exceeds 180° . This restriction is justified by the fact that straight-line motion is locally optimal in free space, and a shortest path only changes direction when forced by obstacle geometry. Such constraints arise precisely at protruding corners of obstacles or boundaries that occlude direct visibility, which are exactly the extremities. Vertices that do not introduce new visibility constraints will never serve as necessary turning points and are therefore not required as intermediate way-points. Figure 2b shows the resulting optimized visibility graph constructed solely over extremities, which is significantly sparser than the full visibility graph while still encoding all shortest paths.

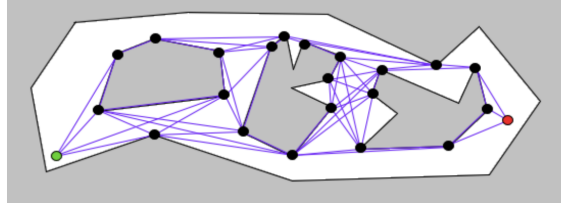
A key practical advantage of this formulation is that the optimized visibility graph depends only on the static geometry of the environment and can therefore be precomputed once and reused across all path-planning queries. For a specific shortest-path query, only two additional vertices, corresponding to the start and goal position, must be added to the graph, along with edges connecting them to the visible extremities. Shortest path planning then reduces to a standard weighted graph search problem, to which classical algorithms such as Dijkstra’s algorithm can be applied directly. In practice, *ExtremityPathFinder* employs the A* algorithm, a heuristic-guided variant of Dijkstra’s algorithm that prioritizes exploration toward vertices estimated to be closer to the goal (by Euclidean distance) while preserving optimality.



(a) Full visibility graph constructed over all boundary and obstacle vertices.



(b) Optimized visibility graph restricted to extremities only.



(c) Shortest path query with start (green) and goal (red) added to the optimized graph

Figure 2: Visibility-graph-based shortest path planning in a polygonal environment. The full visibility graph (top) is reduced to an optimized graph over extremities (middle), which is reused across queries. For each query, only the start and goal vertices are added before computing the shortest path (bottom).

Figure 2c illustrates this process for a specific query, showing the start and goal positions added to the precomputed graph.

Monte Carlo Tree Search and Graph Search

Monte Carlo Tree Search (MCTS) is a decision-making algorithm widely used in games and planning problems where the search space is too large or complex to be explored exhaustively. Instead of enumerating all possible action sequences, MCTS performs a best-first search by incrementally constructing a tree of possible future decisions. In this tree, nodes correspond to system states and edges correspond to actions that transition between states.

Each iteration of MCTS follows a four-phase procedure. During the selection phase (phase 1), the search descends through the existing tree by repeatedly selecting actions that balance exploitation of empirically strong choices with exploration of less-visited alternatives, guiding the search towards regions of the tree where additional computation is expected to be the most informative. This process continues until a node is reached for which not all available actions have been added to the tree. At this point, the expansion phase (phase 2) augments the tree by selecting one of these previously unexplored actions, adding it as an outgoing edge from the selected node, and creating the corresponding child node. The newly added node is then evaluated in the simulation phase (phase 3), where its long-term utility is estimated, typically by sampling actions until a terminal state or a predefined depth limit is reached. Finally, during backpropagation (phase 4), the simulation outcome is propagated back along the selected path, updating visit counts and value estimates for each encountered node. By repeatedly applying this process, MCTS allocates increasing computational effort to actions that appear more promising based on accumulated simulation results. As a result, MCTS is an anytime algorithm: its action-value estimates and resulting decision quality improve progressively as additional computation time becomes available [Chaslot et al., 2008].

As mentioned, traditional MCTS models the search space as a tree, implicitly assuming that each distinct sequence of actions leads to a unique state. In many domains, however, this assumption does not hold, as multiple action sequences may converge to the same underlying state. Such state re-encounters are referred to as *transpositions*. When transpositions are ignored, redundant nodes are created in the search tree, fragmenting value estimates and reducing search efficiency. This issue is especially pronounced in continuous navigation problems, where the same physical location at a given timestep can often be reached via many different paths.

Monte Carlo Graph Search (MCGS) addresses this limitation by generalizing MCTS to operate over a directed acyclic graph (DAG) rather than a tree. In this formulation, each unique state is represented by a single node, while multiple incoming edges capture the different action sequences that lead to that state. By explicitly accounting for transpositions, MCGS aggregates rollout statistics across all paths converging on the same state, reducing redundancy and improving learning efficiency [Mulvey, 2024]. As a result, MCGS provides an effective foundation for stealth path planning in continuous environments.

Related Work

This work builds directly on the 2016 master’s thesis by Singh, also supervised by Verbrugge, which investigated heuristic hiding strategies for non-player characters in two-dimensional polygonal game environments [Singh, 2016]. In that work, Singh examined how an agent can remain concealed from a moving observer in environments modelled as polygons with holes, where the observer follows a known, deterministic path and possesses a 360° FoV with unbounded range. Visibility is evaluated at discrete timesteps along the observer’s path, and the resulting shadow regions define the feasible space in which the agent may move while remaining undetected by the observer.

Within this framework, Singh evaluated several computationally inexpensive heuristic strategies for selecting agent movements inside shadow regions, including greedy relocation to the nearest future shadow, selecting positions that lie deepest within the current shadow, and continuously centring the agent within the shadow region using locally defined geometric criteria. These approaches were compared against an optimal clairvoyant strategy that assumes complete foreknowledge of future visibility changes. While the heuristic methods performed reasonably well in some scenarios, particularly in environments with smoothly evolving shadows, Singh found that they frequently failed to approach optimal performance due to their inherently local and reactive nature. In particular, the heuristics lacked the ability to plan proactively over extended time horizons or to reason globally about the structure and evolution of future safe regions.

The limitations identified in this prior work motivate the present research, which seeks to replace purely local heuristic decision-making with a principled, planning-based approach capable of efficiently reasoning over future visibility and movement constraints.

Method

Having established the relevant background, we now describe the problem formulation and algorithmic design choices underlying our approach.

This work adopts a problem setup that closely follows the environment and visibility model introduced by Singh. We consider two-dimensional game levels represented as simple polygons with polygonal holes, where all geometry is defined in continuous space. The environment contains two types of characters: one or more guards, whose motion follows predefined trajectories through the level, and an agent whose objective is to remain hidden from the guards. Guard characters are assumed to possess a 360° FoV with unbounded range and to move along deterministic paths that are discretized into uniform timesteps. At each timestep, the visibility polygon of each guard is computed, inducing corresponding shadow regions within the map. The agent’s performance is then evaluated by determining whether its position lies within the shadow region at each timestep, thereby measuring its ability to avoid detection over the duration of the guards’ trajectories.

Level Representation

Each level passed as input to the algorithm is specified using a structured JSON file that encodes the static geometry of the environment, agent parameters, and deterministic guard trajectories. At runtime, the level

file is parsed and validated before being used to construct an internal map representation on which the algorithm operates. The semantics and constraints associated with each component of the representation are described below.

Boundary. The `boundary` field defines the outer boundary of the level as a simple polygon specified by an ordered list of vertices in continuous two-dimensional space. The vertices must be listed in counterclockwise (CCW) order.

Obstacles. The `obstacles` field contains a collection of polygonal obstacles embedded within the level boundary. Each obstacle is represented as a simple polygon, with vertices listed in clockwise (CW) order.

Player Parameters. The agent is modelled as a disk with radius `player_radius`, which is used for collision checking against both the level boundary and obstacles. The `player_speed` parameter specifies the maximum distance the agent may move in a single timestep. An optional parameter, `player_start`, defines the agent’s initial position, which is assumed to lie within the walkable area of the level.

Guard Paths. Guard motion is specified by the `guards` field, which contains a list of guard definitions. Each guard is associated with an ordered list of `positions`, where each position corresponds to the guard’s location at a discrete timestep along its deterministic trajectory.

Map Construction

After parsing the JSON input, the level specification is converted into an internal geometric map representation that supports fundamental polygonal operations such as containment queries, set operations (e.g., union and difference), and geometric buffering, as introduced in the background section. Using the Shapely library, the boundary and obstacle polygons are validated and assembled into a single polygonal environment with holes, representing the walkable area of the level [Gillies et al., 2025].

The polygonal environment is first converted into a visibility environment using the *VisiLibity* library, enabling robust visibility polygon computation from arbitrary guard positions [Obermeyer and Contributors, 2008]. At each timestep along the guards’ deterministic trajectories, a visibility polygon is computed independently for each guard based on its position at that timestep. When multiple guards are present, their individual visibility polygons are combined using Shapely’s polygonal union operation to produce the overall visible region for that timestep, which is represented as a (multi)polygon and post-processed to ensure geometric validity. The corresponding shadow region is then obtained by computing the set difference between the walkable area of the level and the combined visibility region, yielding the set of locations in which the agent remains unobserved. This procedure is repeated for all timesteps along the guard paths, and the resulting visibility and shadow polygons are precomputed and stored for use during planning, allowing the planner to query safe regions efficiently without recomputing visibility during search.

To account for the agent’s physical extent, the player is modelled as a disk with a fixed radius rather than as a point. Rather than explicitly incorporating this disk geometry into all subsequent path-planning computations, the environment geometry is pre-processed using polygon buffering operations. Specifically, each obstacle polygon is expanded outward by the player’s radius, while the outer boundary of the level is contracted inward by the same amount. This pre-processing step defines a reduced walkable area for path-planning, ensuring that all planned trajectories respect the agent’s physical size and avoid collisions with obstacles, the boundary, or infeasibly narrow passages. This approach simplifies geometric reasoning while preserving correctness and is a standard technique in motion planning for disk-shaped agents [LaValle, 2006].

From the buffered walkable region, a visibility-graph-based path-planning representation is constructed as described in the background section. We use the *ExtremityPathFinder* library to build an optimized visibility graph over the static environment geometry, which is constructed once at initialization and reused throughout planning [Schneider, 2020]. This representation enables efficient shortest-path queries between candidate positions while guaranteeing collision-free motion for the disk-shaped agent.

Lastly, to control redundancy and ensure numerical robustness when operating in continuous space, we introduce a spatial pruning tolerance that specifies the minimum resolution at which two positions are

considered distinct. This tolerance is defined as a fixed fraction of the overall size of the environment, using the square root of the buffered walkable area as a characteristic length scale. Formally, the pruning tolerance ε is given by

$$\varepsilon = \alpha\sqrt{A},$$

where A denotes the area of the buffered walkable region, and α is a dimensionless scaling constant. In all experiments presented in this work, we set $\alpha = 0.02$, yielding a pruning tolerance of approximately 0.2 for each evaluated level. This tolerance is used throughout the algorithm to discretize continuous positions, identify equivalent states, and reduce redundant computation; its specific roles are described in detail where they arise in subsequent sections.

Kernel Computation

At each timestep, approximated kernel points are extracted from shadow regions using a recursive polygon insetting algorithm based on negative buffering. Given a shadow geometry S , Algorithm 1 repeatedly applies an inward offset that progressively shrinks the shape while preserving its overall topology. As the polygon is inset, narrow regions are eliminated first, and sufficiently large regions may split into multiple disconnected components, each of which is processed independently and whose resulting kernels are combined.

Algorithm 1: FINDKERNELS: Recursive Kernel Extraction via Polygon Insetting

```

Input: Geometric shape  $S$ , recursion depth  $d$ 
Output: Set of kernel points annotated with depth
// Base cases
if  $S$  is empty then
  | return  $\emptyset$ ;
if  $S$  is a point then
  | annotate  $S$  with depth  $d$ ;
  | return  $\{S\}$ ;
if  $S$  is a line segment then
  |  $K \leftarrow \{\text{endpoints of } S, \text{midpoint of } S\}$ ;
  | annotate each point in  $K$  with depth  $d$ ;
  | return  $K$ ;
// If multipolygon, recursively process each component
if  $S$  is a multi-polygon then
  |  $K \leftarrow \emptyset$ ;
  | foreach sub-polygon  $S_i \in S$  do
  | |  $K \leftarrow K \cup \text{FINDKERNELS}(S_i, d + 1)$ ;
  | return  $K$ ;
// Otherwise, perform the polygon inset
 $\delta \leftarrow \max(0.01 \cdot \sqrt{\text{area}(S)}, 0.01)$ ;
 $S' \leftarrow \text{inset}(S, -\delta)$ ;
// Check if it's a base-case before we lose the polygon
if  $S'$  is empty or  $S' = S$  then
  |  $p \leftarrow$  representative interior point of  $S$ ;
  | annotate  $p$  with depth  $d$ ;
  | return  $\{p\}$ ;
// If not, make a recursive call
return  $\text{FINDKERNELS}(S', d + 1)$ ;

```

The recursion handles several base cases explicitly. If the input shape is empty, the procedure returns an empty kernel set. If the shape collapses to a single point, that point is returned as a kernel. If the shape collapses to a line segment, kernel points are extracted at both endpoints and at the midpoint of the segment.

If the shape is a multi-polygon, the algorithm recursively processes each component polygon and aggregates the kernels returned from each subproblem.

Otherwise, for a valid polygonal region, the algorithm computes an adaptive inset distance

$$\delta = \max(0.01 \cdot \sqrt{\text{area}(S)}, 0.01),$$

and applies a negative buffer of magnitude δ . The recursion continues until further inseting would either eliminate the geometry or leave it unchanged. In this termination case, a representative interior point is returned as the kernel, using the centroid when it lies within the polygon and a representative interior point otherwise.

Each extracted kernel point is assigned a depth value equal to the number of recursive inseting operations required to reach it. This depth serves as an approximate measure of how deeply embedded the kernel lies within the original shadow region: points with larger depth values persist through more inseting steps and, therefore, correspond to locations that are farther from visibility boundaries and more robust to small changes in observer position. Furthermore, at every stage of the recursion, the candidate kernel points produced are validated against the buffered walkable region, and any point outside this region is discarded. Consequently, the algorithm never returns kernel locations that are geometrically unreachable by the agent.

For kernel extraction, Algorithm 1 is called on two closely related shadow representations at every timestep, illustrated in Figure 3.

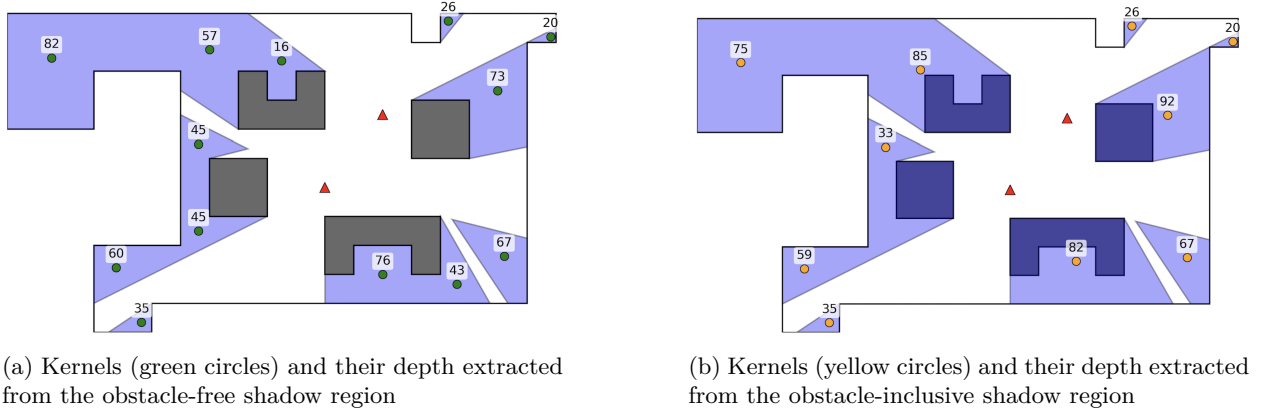


Figure 3: Comparison of kernel extraction from the two shadow representations at a fixed timestep, with shadows shown in blue and guard positions indicated by red triangles.

The first representation corresponds to the unmodified shadow region, capturing the global structure of concealment induced solely by line-of-sight constraints. Applying Algorithm 1 to this region yields kernel points that summarize the spatial organization of hidden space, providing candidate positions representative of a large, contiguous area of invisibility. As illustrated in Figure 3a, these kernels are distributed throughout the shadow region and serve as anchors for reasoning about movement within broad areas of cover.

The second representation augments the shadow region by reintroducing interior obstacles into the shadow geometry, effectively treating obstacle interiors as shadowed space. Because kernel extraction inherently favours large, convex regions, omitting obstacles can bias kernels toward the geometric centres of broad shadows, overlooking smaller but strategically important hiding pockets near obstacles. In practice, as illustrated in Figure 3b, incorporating obstacles into the shadow geometry counteracts this effect by preserving local concavities and obstacle-induced recesses, allowing kernels to form near obstacle boundaries and capture these tactically valuable positions. Since obstacle interiors are treated as shadowed space in this representation, the inseting process may cause the shadow geometry to collapse to points that lie inside obstacles. The walkability constraint introduced earlier is therefore essential: any such points are rejected and not returned as kernels.

For each timestep, the kernel sets extracted from the obstacle-free and obstacle-aware shadow representations are merged into a single candidate set. To prevent redundancy, kernel points with the same location and depth are merged, ensuring that each distinct kernel is represented only once. The resulting unified kernel

set provides a compact yet expressive summary of the concealment structure at that timestep, integrating both globally safe regions and locally induced hiding opportunities. This combined representation forms the primary set of candidate target locations used by the planner when generating and evaluating actions in the MCGS.

Monte Carlo Graph Search

Once the geometric structure of the environment has been constructed and concealment features have been extracted in the form of shadow regions and kernel points, the remaining challenge is to plan a sequence of agent positions over time. Ideally, this sequence would remain fully hidden from all guards for the entire planning horizon. In practice, however, complete concealment is not always achievable due to map geometry, guard motion, and movement constraints. As a result, the planning problem is formulated as an optimization task: selecting a path that maximizes the fraction of timesteps during which the agent remains in shadow. In this work, performance is measured as the percentage of timesteps spent hidden over the full horizon, providing a continuous objective that rewards both long-term concealment and graceful recovery from brief exposures.

To address this planning problem, we employ Monte Carlo Graph Search (MCGS), with the search graph rooted at depth 0, corresponding to the agent’s position at timestep 0. The agent’s initial position in our implementation is an optional input parameter. When an explicit start position is not provided, the agent’s initial location is sampled from the shadow region at timestep 0. Because this shadow region often consists of multiple disconnected shadow polygons, sampling is biased toward larger feasible regions by selecting a sub-polygon with probability proportional to its area, with candidate positions accepted only if they lie within the buffered walkable area.

State and Graph Representation

Each node in the graph corresponds to a unique agent state defined by its spatial position and discrete timestep, with node depth equal to the timestep. The search structure is represented as a directed acyclic graph to support transpositions, allowing multiple paths to lead to the same state. Nodes store visit counts and value estimates required for MCGS, while edges represent feasible movement actions between states and maintain their own statistics for action selection. This separation enables efficient aggregation of experience across transpositions while preserving action-level information for decision-making.

Potential Move Generation

For a node at depth t , potential moves correspond to candidate agent positions at timestep $t + 1$ and are generated lazily upon first visit, then cached for reuse. Nodes at the maximum search depth are treated as terminal and do not generate outgoing moves. The maximum search depth is determined by the length of the guard trajectories; if T denotes the total number of discrete timesteps in each guard’s path, the maximum allowable depth is set to $T - 1$, since shadow geometry is defined only for timesteps 0 through $T - 1$ and nodes at depth $T - 1$ therefore admit no further expansion.

Potential moves for a node at depth t are primarily derived from kernel points extracted from the shadow geometry at timestep $t + 1$. For each kernel, a shortest path is computed from the position stored in the node, and a candidate move is obtained by advancing along this path by at most the agent’s maximum step length. To improve robustness when kernel guidance is sparse or restrictive, the move set may be augmented with a small number of brute-force moves generated by stepping in evenly spaced directions around the agent, validated against the map’s walkable area. If no valid moves are available, a stationary action is included to ensure that every non-terminal node admits at least one successor. Finally, generated moves are randomly shuffled to avoid bias in expansion order.

Main Search Loop and Phases

The MCGS algorithm proceeds by repeatedly executing a main search loop, shown in Algorithm 2, that incrementally refines the search graph. Each iteration follows the standard four-phase MCTS procedure, adapted to operate over a directed acyclic graph with shared states and edge-level statistics.

Algorithm 2: RUN: Main Monte Carlo Graph Search Loop

Input: Time budget $t_{\max}$ **Output:** Search graph populated until convergence $t_{\text{start}} \leftarrow \text{current time};$ **while** *true* **do** $(v, P) \leftarrow \text{SELECT}();$ *// v is selected leaf node, P is list of traversed edges* $(u, e) \leftarrow \text{EXPAND}(v);$ **if** $(u, e) \neq \text{None}$ **then** append e to P ; $R \leftarrow \text{EVALUATE}(u);$ **else** $R \leftarrow v.\text{score} ;$ *// Node fully expanded; reuse stored evaluation* $\text{BACKPROPAGATE}(u, P, R);$ $\text{elapsed} \leftarrow \text{current time} - t_{\text{start}} ;$ $C \leftarrow \text{GETPATHMAXVISITS}();$ **if** $\text{elapsed} \geq t_{\max}$ **and** $|C| \geq \text{max_depth}$ **then** **return**;

The search is primarily bounded by a fixed time budget, allowing the algorithm to perform sufficient computation before returning a solution. In all experiments, the search must run for at least 60 seconds. In addition, a structural termination condition is imposed to ensure convergence over the full planning horizon. After each iteration, the algorithm extracts its current consensus solution by greedily following, from the root, the outgoing edge with the highest visit count. Selecting the most visited action as the algorithm’s current best estimate of an optimal trajectory follows standard practice in MCTS, where visit counts are interpreted as a measure of statistical confidence in an action’s quality [Chaslot et al., 2008]. Once this most frequently visited path reaches length $T - 1$, corresponding to a complete trajectory over the guard timeline, the search is considered to have converged.

With this execution loop and termination logic in place, we now describe the individual phases that comprise a single iteration of the MCGS algorithm.

Selection: The selection phase traverses the existing search graph to identify a node suitable for expansion. Starting from the root, the algorithm repeatedly selects outgoing edges based on accumulated search statistics, descending through the graph until either a node eligible for expansion is reached or a terminal node is encountered.

Because the underlying state space is continuous, the number of potential actions at each node can be large. To control the branching factor, selection incorporates *progressive widening*, which limits the number of outgoing edges a node may expand as a function of its visit count. For a node with visit count N , the maximum number of expandable children is given by

$$n_{\max}(N) = \min(|\mathcal{A}|, \max(1, \lfloor k \cdot N^\alpha \rfloor)),$$

where $|\mathcal{A}|$ is the number of available moves and $k > 0$, $\alpha \in (0, 1)$ are tunable parameters. This scheme encourages deep exploration early in the search while gradually introducing additional actions as confidence in the region increases. Across all experiments, we found that setting $k = 1$ and $\alpha = 0.3$ provided a robust trade-off between depth and exploration, consistently allowing the search to converge to full-length solutions around the allotted 60-second time budget. These values were therefore used throughout this work.

For a node encountered during selection, expansion is permitted only if three conditions are satisfied: the node has at least one potential move, it is not already fully expanded, and the number of its outgoing edges is below the limit imposed by progressive widening. If any of these conditions hold, or if the node has no outgoing edges at all, selection terminates, and the node is returned for expansion.

Otherwise, selection continues by choosing the outgoing edge that maximizes an Upper Confidence Bound (UCB) score. For an edge e with visit count N_e and accumulated value V_e , originating from a parent node

with visit count N_p , the UCB score is defined as

$$\text{UCB}(e) = \begin{cases} \infty, & N_e = 0, \\ \frac{V_e}{N_e} + c\sqrt{\frac{\ln(N_p + 1)}{N_e}}, & N_e > 0, \end{cases}$$

where $c = \sqrt{2}$. This selection rule directs computational effort toward the most promising transitions while ensuring that alternative actions continue to be explored.

Expansion: Once selection identifies a node eligible for growth, the expansion phase adds at most one new outgoing edge from its cached set of potential moves. To prioritize concealment, expansion proceeds in two ordered passes over the candidate moves, which are randomly shuffled in advance. In the first pass, only moves that keep the agent hidden at the next timestep are considered, and the first valid, previously unexplored hidden move is expanded. If no such move is available, a second pass allows transitions that result in temporary visibility, enabling progress in environments where brief exposure is unavoidable. In both cases, candidate moves are discarded if they are not within the buffered walkable area or are within the pruning tolerance (ϵ) of an existing outgoing edge. If no valid move can be added, the node is marked as fully expanded. This strategy limits redundant expansion while favouring hidden trajectories whenever possible, yet retains the flexibility needed to complete full-length plans.

Simulation: After a new transition is added during expansion, the resulting child node is evaluated to estimate the quality of the corresponding state. The evaluation function depends on whether the agent is visible or hidden at the node’s timestep.

If the agent is visible, the node score is computed as

$$\text{Score}_{\text{visible}} = \theta e^{-\tau d_s} + p + \beta (1 - e^{-\tau d_g}) + \gamma e^{-\tau d_k} + \eta \left(\frac{n_d}{T - 1} \right)^2 + \lambda r,$$

where d_s denotes the minimum Euclidean distance from the agent’s position to the shadow region at the current timestep. This term encourages positions that are closer to regaining concealment, while the constant $p < 0$ applies an explicit penalty to visible states.

If the agent is hidden, the node is instead given by

$$\text{Score}_{\text{hidden}} = \delta (1 - e^{-\tau d_v}) + \beta (1 - e^{-\tau d_g}) + \gamma e^{-\tau d_k} + \eta \left(\frac{n_d}{T - 1} \right)^2 + \lambda r,$$

where d_v is the minimum Euclidean distance to the visible region at the current timestep. In this case, larger distances correspond to deeper concealment and therefore yield higher scores.

The remaining terms are shared between both formulations. The guard-distance term uses d_g , the average shortest-path distance from the agent to each guard at the current timestep, encouraging positions that remain farther from guards. The kernel-distance term uses d_k , the average shortest-path distance from the agent to the k deepest kernel points at the next timestep, biasing the search toward positions that are well placed to reach robust hiding regions in the near future.

All distance-based terms employ an exponential decay, with a shared decay rate τ , normalizing each contribution to the range $[0, 1]$, making the relative weights invariant to map scale, and the constants for each term directly comparable. In all experiments, we set $\tau = 0.1$, approximately half the pruning tolerance ϵ , which empirically produced well-distributed scores across the full range $[0, 1]$.

The depth bonus term depends on the node depth n_d relative to the maximum search depth $T - 1$ and incentivizes exploration of longer trajectories, helping the search converge to full-length solutions in environments with long planning horizons. Finally, the rollout term r provides a lightweight estimate of future survivability by evaluating whether the agent’s current position is hidden after a fixed, tunable number of subsequent timesteps, referred to as the rollout depth (ϕ). This term is binary: $r = 1$ if the agent remains concealed for the entire rollout horizon, and $r = 0$ otherwise. It supplies the evaluation function with a coarse but forward-looking signal that biases the search toward states that are likely to remain hidden in the future.

Overall, the simulation phase provides a structured heuristic estimate of state quality that integrates current concealment, spatial context, and future survivability, supplying MCGS with a forward-looking signal for prioritizing promising branches of the search graph.

Backpropagation: During backpropagation, the evaluation score obtained at the expanded node is propagated backward along the selection path, updating visit counts and accumulated values for both nodes and edges encountered along the trajectory. This process ensures that states and transitions that consistently lead to high-quality outcomes accrue higher average values over time, while less effective choices are progressively de-emphasized. By aggregating statistics across shared states and edges, MCGS naturally integrates information from multiple action sequences that converge to the same state, reinforcing promising regions of the search graph and enabling efficient learning under transpositions.

Results

To evaluate the effectiveness of the proposed visibility-aware MCGS planner, we conducted experiments across three map environments of increasing complexity, shown in Figure 4.

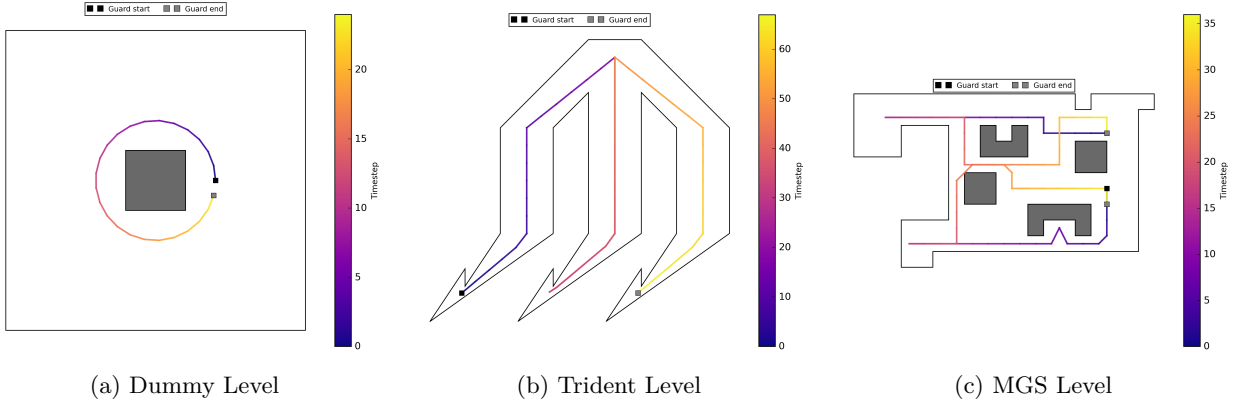


Figure 4: Guard trajectories in each of the experimental levels, colour-coded by timestep.

Dummy Level. The Dummy level, shown in Figure 4a, is a highly simplified environment designed as a sanity check for the planner. It consists of a single central obstacle and a single guard following a circular trajectory around it. As the colour-coded path indicates, the guard’s visibility sweeps uniformly around the obstacle over time, producing a predictable and symmetric pattern of shadow regions. In this setting, the optimal behaviour is straightforward: the agent should mirror the guard’s motion by remaining on the opposite side of the obstacle, effectively orbiting the central geometry to stay continuously concealed. This level allows the planner’s basic reasoning about visibility, motion, and timing to be validated against an analytically obvious solution.

Trident Level. The Trident level, shown in Figure 4b, is a deliberately structured environment intended to test the planner’s ability to reason over long-term consequences rather than purely local concealment. The guard follows a deterministic path that sequentially explores three narrow corridors, as indicated by the colour progression along each prong. Although remaining hidden locally is often possible, the globally optimal strategy requires foresight: the agent must initially move to the right corridor, then exploit a brief occlusion, when the guard moves behind the central prong, to traverse across the top of the map and reposition into the left corridor before visibility returns. This environment highlights the importance of planning for future shadow regions, rather than greedily optimizing for immediate concealment.

Metal Gear Solid-Inspired Level. The final environment, shown in Figure 4c, is the most complex and realistic scenario, inspired by a level from *Metal Gear Solid* (MGS). Multiple guards follow intersecting, time-varying trajectories, producing visibility regions that fragment, overlap, and reconfigure over time. Shadows in this level are often narrow, transient, and spatially disconnected, requiring the agent to exploit short-lived hiding opportunities near obstacles and walls. This level stresses the scalability and robustness of the approach under realistic stealth conditions, where the search space is large and the geometry of concealment changes rapidly across timesteps.

To reduce sensitivity to initial conditions, each experimental configuration was evaluated from multiple starting positions within each level: six on the Dummy level, eight on the Trident level, and ten on the MGS level. Starting positions were selected to span a range of spatial configurations with varying proximity to guards, obstacles, and shadow regions, while ensuring the agent initially began in concealment. Evaluating performance across multiple starting positions provides a more representative measure of overall behaviour and allows us to assess robustness to local geometric variation, which is particularly important in stealth environments where small positional changes can substantially affect visibility outcomes.

For clarity and conciseness, the main body of this paper reports only aggregated results, averaged across starting positions for each experimental condition and level. Complete per-run results, including metrics for every individual starting position and every experimental configuration, are provided in Appendix A.

Experiment 1: Visibility Penalties

The first experiment investigates the impact of the visibility penalty term in the node evaluation function, which controls how strongly the planner is discouraged from selecting states in which the agent is visible to a guard. To examine how this parameter affects solution quality and risk sensitivity, we varied the visibility penalty p across three values, $p \in \{-1, -10, -100\}$, while holding all other hyperparameters fixed.

Table 1 reports the resulting average concealment performance for each level, measured as the percentage of timesteps during which the agent remained hidden and averaged across all starting positions. Full results in Appendix A, Table 5.

Table 1: Effect of visibility penalty (p) on average concealment performance across levels. Results are averaged over all starting positions for each level.

Level	Total Timesteps	p	Avg Runtime (s)	Avg Time Hidden (%)
Dummy	24	-1	60.00	94.44
		-10	60.00	94.44
		-100	60.00	95.14
Trident	67	-1	79.52	62.87
		-10	60.45	99.07
		-100	64.44	93.66
MGS	36	-1	126.80	62.22
		-10	68.88	94.72
		-100	81.91	97.22

On the Trident level, the results in Table 1 show that setting $p = -1$ leads to poor concealment performance, with the agent remaining hidden for only 62.87% of the planning horizon. In this regime, the visibility penalty is too weak relative to other components of the score, such as kernel proximity and rollout value, causing the search to favour locally advantageous states even when they lead to repeated exposure.

At the other extreme, increasing the penalty to $p = -100$ improves concealment relative to $p = -1$ but still underperforms the intermediate setting, achieving 93.66% time hidden. Certain runs, like starting from the bottom of the middle corridor, require a very narrow sequence of precise early moves to safely reach the right-hand corridor. The vast majority of alternative actions in this region lead to repeated early detections, each incurring the large negative penalty. As a result, the average value of edges progressing toward the right corridor is heavily depressed by these frequently penalized outcomes, even though a small subset of trajectories eventually leads to superior long-term concealment. During selection, these low average values

cause the entire subtree to be systematically deprioritized. Instead, the search converges toward the middle corridor, where the agent can remain hidden for most timesteps and incur only a single detection, yielding higher short-term rewards despite being globally suboptimal in terms of long-term survivability.

In contrast, the intermediate setting $p = -10$ yields the best performance on the Trident level, achieving 99.07% time hidden. This value is large enough to ensure that concealment dominates the evaluation function, while still allowing the search to explore narrowly constrained, high-reward trajectories that involve limited early exposure. This balance enables effective long-horizon reasoning and consistent convergence to the known optimal solution.

For the Dummy and MGS levels, Table 1 shows a monotonic improvement in concealment as the magnitude of the visibility penalty increases. These environments do not require deliberate early exposure to achieve long-term concealment, and stronger penalties simply reinforce the desired behaviour of remaining hidden. In particular, on the MGS level, increasing p from -1 to -100 improves average concealment from 62.22% to 97.22%, indicating that strong visibility aversion is beneficial in complex, highly dynamic stealth scenarios.

Overall, the results in Table 1 demonstrate that the visibility penalty must be large enough to dominate other components of the evaluation function in order to meaningfully prioritize concealment. However, over-penalizing visibility can suppress exploration of strategically necessary, high-risk actions in certain environments. These findings suggest that the appropriate strength of the visibility penalty depends on the structural properties of the environment: strong penalties are effective when concealment is consistently achievable, while scenarios that demand deliberate risk-taking benefit from a more moderate penalty that preserves exploratory flexibility.

Experiment 2: Kernel-Based Bias

This experiment examines how strongly and how broadly the search should be biased toward the deepest shadow regions in the environment. Specifically, we study the effect of (i) the weight assigned to the kernel distance term in the evaluation function (γ) and (ii) the number of deepest kernels included (k) when computing this score. Table 2 summarizes the results across all three levels. Full results in Appendix A, Table 6.

Table 2: Effect of kernel distance weight (γ) and number of kernels (k) included in the score. Results are averaged over all starting positions for each level.

Level	Total Timesteps	γ	k	Avg Runtime (s)	Avg Time Hidden (%)
Dummy	24	1	1	60.00	94.44
		1	3	60.00	94.44
		2	1	60.00	94.44
		4	1	60.00	94.44
Trident	67	1	1	60.45	99.07
		1	3	61.03	98.13
		1	5	62.25	96.64
		2	1	60.00	98.32
		4	1	60.00	98.13
MGS	36	1	1	109.96	96.11
		1	3	114.04	95.83
		1	5	133.08	96.94
		2	5	125.35	96.67
		4	5	119.23	96.11

On the Dummy level, varying either the kernel weight or the number of kernels does not affect performance, as shown in Table 2. This is expected, as the environment contains a single dominant concealment region and does not require nuanced long-term planning.

In contrast, both the Trident and MGS levels exhibit clear sensitivity to these parameters. On the Trident level, when the score incorporates multiple kernels, the planner is implicitly encouraged to remain centrally

positioned with respect to all candidate hiding regions. In practice, this causes the agent to linger in the middle corridor, where it is locally well placed relative to several kernels, but the agent becomes visible as the guard traverses the level, resulting in reduced long-term concealment. As shown in Table 2, increasing the number of kernels from one to three reduces the average time hidden from 99.07% to 98.13%, with a further drop to 96.64% when five kernels are considered. In contrast, restricting the score to the single deepest kernel allows the agent to commit to one robust shadow region, typically one of the side corridors, yielding the strongest overall performance.

Increasing the kernel-distance weight further exacerbates this effect. When the kernel term becomes more dominant relative to the visibility penalty, proximity to kernels increasingly outweighs concealment considerations, causing the planner to favour positions that are locally close to deep kernels even when they are visible. This trend is reflected in the reduction in average time hidden from 99.07% with $\gamma = 1$ to 98.13% with $\gamma = 4$, as reported in Table 2.

However, we see the opposite effect on the MGS level where performance improves as the score incorporates a larger number of kernels, as shown in Table 2. When planning toward only one or three kernels, average concealment performance is lower, at 96.11% and 95.83% respectively, indicating that the planner becomes overly focused on a small subset of locally deep shadow regions. This narrow focus reduces flexibility in a highly dynamic environment, where visibility regions frequently fragment and reconfigure. Including five kernels produces the best results, achieving the highest average time hidden at 96.94%. In this setting, the score reflects proximity to multiple distinct concealment regions distributed across the map, encouraging positions that are not only safe in the present but also well positioned to reach several future hiding opportunities.

Increasing the kernel-distance weight on the MGS level exhibits a similar trend to that observed on the Trident level. While moderate weighting maintains strong performance, larger kernel weights slightly reduce concealment - from 96.94% with $\gamma = 1$ to 96.11% with $\gamma = 4$ - by amplifying the influence of kernel proximity relative to explicit visibility costs.

Overall, these results indicate that the kernel score should act as a soft bias rather than a primary driver of decision-making. A small kernel weight allows the heuristic to break ties between otherwise safe states without overpowering concealment objectives. However, increasing the kernel-distance weight beyond a moderate value consistently degrades performance, as the evaluation becomes overly greedy and prioritizes short-term proximity to deep kernels at the expense of sustained concealment. On the other hand, the appropriate number of kernels depends strongly on the structure of the environment. In maps with a single dominant hiding region, restricting the score to a single deep kernel encourages commitment to a robust shadow and yields the best results. In contrast, environments with multiple, distributed, and rapidly changing concealment regions benefit from incorporating a larger set of kernels, which provides a more representative summary of future hiding opportunities. In practice, the number of kernels included in the score should reflect the diversity of meaningful concealment regions in the map, while remaining small enough to limit computational overhead. As shown in Table 2, increasing the number of kernels generally leads to higher average runtime, particularly in more complex environments such as the MGS level. This increase arises because each additional kernel introduces extra shortest-path queries during evaluation. Consequently, although planning toward more kernels can improve robustness in environments with distributed hiding regions, the smallest number of kernels that adequately captures the map’s concealment structure is preferred in order to balance solution quality against computational cost.

Experiment 3: Rollout-Based Foresight

The third experiment investigates the role of the rollout term in the node evaluation function. This experiment explores two related questions: (1) how heavily the rollout score should be weighted relative to other components of the evaluation function, and (2) how far into the future the rollout should extend to provide useful guidance without introducing excessive noise or computational overhead. To study these effects, we varied both the rollout weight λ and the rollout depth ϕ while holding all other hyperparameters fixed. A summary of the results is shown in Table 3. Full results in Appendix A, Table 7.

On the Dummy level, varying the rollout weight and depth has no measurable effect on performance: all configurations converge to the same average concealment rate. This behaviour is expected, as the optimal solution on this level does not require long-term foresight or temporary sacrifices in concealment, and

Table 3: Effect of rollout weight (λ) and rollout depth (ϕ) on concealment performance across levels. Results are averaged over all starting positions for each level.

Level	Total Timesteps	λ	ϕ	Avg Runtime (s)	Avg Time Hidden (%)
Dummy	24	1	7	60.00	95.14
		2	15	60.00	95.14
		2	30	60.00	95.14
		4	7	60.00	95.14
Trident	67	1	30	67.04	93.66
		2	30	62.52	94.40
		4	7	60.86	94.59
		4	15	60.16	95.34
		4	30	60.45	99.07
MGS	36	1	7	107.17	95.56
		2	7	81.91	97.22
		2	15	93.12	95.83
		2	30	80.22	93.89
		4	7	70.55	97.78

immediate visibility information is sufficient to guide the search.

In contrast, both the Trident and MGS levels exhibit a strong dependence on the rollout term, though in qualitatively different ways. On the Trident level, increasing the rollout depth consistently improves performance when the rollout is weighted sufficiently. As shown in Table 3, the highest concealment rate (99.07%) is achieved with a large rollout depth ($\phi = 30$) and a strong rollout weight ($\lambda = 4$). Shallower rollouts fail to capture delayed exposure events, causing the search to favour states that appear safe in the short term but lead to unavoidable detection later. Deeper rollouts, when appropriately weighted, allow the algorithm to account for these delayed consequences and select trajectories that are globally safer over the full horizon.

The MGS level exhibits the opposite trend with respect to rollout depth. While increasing the rollout weight improves performance across all tested depths, deeper rollouts consistently degrade performance relative to shallow ones. The best results on this level are obtained with a shallow rollout depth ($\phi = 7$) and a strong rollout weight ($\lambda = 4$), yielding an average concealment rate of 97.78%. As rollout depth increases, performance decreases despite the additional lookahead. This indicates that in highly dynamic environments, long-term visibility outcomes are weakly correlated with near-term safety. In such cases, deeper rollouts introduce noise into the evaluation function rather than providing a useful predictive signal, while also increasing computational cost.

Across both complex levels, one consistent trend emerges: rollout values must be weighted heavily in order to influence decision-making meaningfully. Configurations with low rollout weights underperform even when rollout depth is sufficient, as immediate rewards such as kernel proximity or guard distance dominate the score. Conversely, when the rollout is strongly weighted, the planner is willing to temporarily accept locally worse states in exchange for improved long-term concealment.

Overall, these results demonstrate that rollouts are a critical component of the evaluation function, but their effective use depends on the temporal structure of the environment. In settings where visibility changes slowly and future danger is predictable, deeper rollouts provide valuable foresight and significantly improve performance. In highly dynamic environments, however, shallow rollouts are more effective, as deeper lookahead adds variance without improving predictive accuracy. In all cases, the rollout term should be weighted strongly enough to influence decisions, but its depth should be chosen conservatively to balance foresight, noise, and computational efficiency.

Experiment 4: Brute-Force Action Augmentation

In the final experiment, we examine the impact of augmenting the action space with brute-force movement options. In previous experiments, the agent’s actions were exclusively heuristic-driven, consisting of moves

along shortest paths toward shadow kernels. While this restriction focuses the search on semantically meaningful actions, it may also limit exploration if the heuristic fails to capture locally advantageous moves. To assess this trade-off, we introduce an optional set of brute-force actions and examine whether the added exploratory flexibility improves performance or instead degrades search efficiency by increasing the branching factor. For each level, we compare performance with brute-force moves enabled and disabled, holding all other hyperparameters fixed. Results are summarized in Table 4. Full results in Appendix A, Table 8

Table 4: Effect of brute-force action augmentation on concealment performance and runtime. Results are averaged over all starting positions for each level.

Level	Total Timesteps	Brute-Force Moves	Avg Runtime (s)	Avg Time Hidden (%)
Dummy	24	TRUE	60.00	93.06
		FALSE	60.00	95.14
Trident	67	TRUE	96.02	96.83
		FALSE	60.45	99.07
MGS	36	TRUE	98.92	95.56
		FALSE	81.91	97.22

Across all three levels, enabling brute-force actions consistently increases runtime while reducing, or failing to improve, concealment performance, as shown in Table 4. On the Dummy level, brute-force augmentation slightly degrades performance: the average time hidden drops from 95.14% without brute-force moves to 93.06% with them, while runtime remains unchanged. This indicates that the heuristic-guided action set is already sufficient for optimal behaviour, and additional unguided actions only introduce unnecessary noise into the search.

The negative effects are more pronounced on the Trident and MGS levels. On the Trident level, enabling brute-force moves increases the average runtime from 60.45 to 96.02 seconds, while reducing the average time hidden from 99.07% to 96.83%. Similarly, on the MGS level, brute-force augmentation increases runtime from 81.91 to 98.92 seconds and lowers concealment performance from 97.22% to 95.56%. These results indicate that the added exploratory flexibility comes at a substantial computational cost without corresponding gains in solution quality.

This behaviour can be attributed to the increased branching factor introduced by brute-force actions. Under a fixed time budget, expanding the action space forces the search to distribute simulations across many additional branches. Because many of the brute-force moves are locally reasonable but strategically unimportant, they often receive favourable short-term evaluations and dominate early selection. As a result, the search expends a significant fraction of its computational budget exploring shallow, locally safe deviations, rather than refining the narrow, high-precision action sequences required for optimal long-term concealment.

While isolated cases exist in which brute-force actions introduce marginally beneficial local moves absent from the heuristic action set, these gains are inconsistent and outweighed by the overall loss in efficiency and solution quality. In structured stealth environments, where strong domain-informed heuristics are available, restricting the action space to heuristic-guided moves produces a smaller, more focused search graph that better supports long-horizon planning.

Overall, these results demonstrate that brute-force action augmentation is generally detrimental under a fixed computation budget. When reliable heuristic guidance exists, limiting the action space sharpens the search and improves convergence, whereas brute-force actions should be used sparingly, if at all, to avoid overwhelming the planner with low-value alternatives.

Conclusion

This work presented a visibility-aware stealth path planner based on MCGS that explicitly reasons over future visibility, geometric concealment structure, and long-horizon consequences. By combining shadow-region analysis, kernel-based geometric abstractions, and a forward-looking evaluation function within an MCGS framework, the proposed approach moves beyond purely reactive heuristics and enables principled planning in dynamic stealth environments without a predefined goal. Experimental results across three levels of increasing complexity demonstrate that the planner can reliably generate long-lived stealth trajectories,

while also revealing how key design choices, such as visibility penalties, kernel bias strength, rollout depth, and action-space restriction, directly shape planning behaviour and performance.

The results highlight several important insights. Strong but balanced visibility penalties are essential for prioritizing concealment without suppressing necessary risk-taking. Kernel-based heuristics are most effective when used as a soft bias rather than a dominant objective, with the appropriate number of kernels depending on the structure and dynamism of the environment. Rollouts provide critical foresight in long-horizon planning but must be tuned carefully to avoid introducing noise in highly dynamic settings. Finally, restricting the action space to heuristic-guided moves consistently improves both efficiency and solution quality, demonstrating that informed abstraction is preferable to brute-force exploration under fixed computational budgets.

There are several promising directions for future work. A natural extension would be to introduce explicit goal locations, allowing the planner to balance stealth against progress toward an objective and bringing the formulation closer to real stealth-game scenarios. Incorporating distance-to-goal into the evaluation function would enable richer trade-offs between safety and advancement. Another extension would be to allow the agent to remain stationary. Many observed trajectories exhibit unnecessary oscillations due to forced movement at every timestep; introducing a deliberate “wait” action could simplify plans, improve concealment stability, and reduce search complexity. Finally, expanding the experimental suite to include additional levels inspired by real commercial games would provide deeper insight into which parameter settings generalize across environments and which are highly level-specific, further strengthening the practical applicability of the approach.

Overall, this work demonstrates that integrating geometric visibility reasoning with MCGS provides a powerful and flexible foundation for stealth planning, capable of capturing both local hiding opportunities and long-term survival strategies in complex, dynamic environments.

References

- Takao Asano, Tetsuo Asano, Leonidas Guibas, John Hershberger, and Hiroshi Imai. Visibility-polygon search and euclidean shortest paths. In *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*, pages 155–164, 1985. doi: 10.1109/SFCS.1985.65.
- Guillaume Chaslot, Sander Bakkes, Istvan Szita, and Pieter Spronck. Monte-carlo tree search: A new framework for game ai. 01 2008.
- Sean Gillies, Casper van der Wel, Joris Van den Bossche, Mike W. Taves, Joshua Arnott, Brendan C. Ward, et al. Shapely, 2025. URL <https://github.com/shapely/shapely>.
- Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006. Available online at <http://planning.cs.uiuc.edu/>.
- Aaron Mulvey. *Monte Carlo Graph Search and Transpositions in Monte Carlo Tree Search*. 11 2024. doi: 10.13140/RG.2.2.29303.43688.
- K. J. Obermeyer and Contributors. VisiLibity: A C++ library for visibility computations in planar polygonal environments. <http://www.VisiLibity.org>, 2008. v1.
- Franco P. Preparata and Michael Ian Shamos. *Computational Geometry: An Introduction*. Springer, New York, NY, 1985. ISBN 978-0-387-96131-6. doi: 10.1007/978-1-4612-1098-6.
- Florian Schneider. Extremitypathfinder: Shortest path planning in polygonal environments, 2020. URL <https://pypi.org/project/extremitypathfinder/>. Python package, version 2.7.2.
- Navjot Singh. Analyzing simple heuristic hiding strategies for non-player characters in games. Master’s thesis, School of Computer Science, McGill University, Montreal, 2016.

A Full Experimental Results

This appendix presents complete per-run results for all experiments described in the main text. For each experimental condition, results are reported for every starting position and hyperparameter configuration. These tables provide additional quantitative detail beyond the aggregated summaries presented previously. In addition, all experimental runs, including full trajectory animations and frame-by-frame visualizations for every configuration, are available in the accompanying repository at <https://github.com/allisonmeikle/stealth-path-planning/tree/main/final-results>.

Table 5: Full per-run results for Experiment 1 (varying visibility penalties). Each row corresponds to a single starting position.

Level	Start Position	Visibility Penalty (p)	Runtime (s)	Timesteps	Hidden	Total Timesteps	Time Hidden (%)
Dummy	(2.0, 2.0)	-100	60.00	24		24	100.00
Dummy	(2.0, 8.0)	-100	60.00	17		24	70.83
Dummy	(3.0, 3.0)	-100	60.00	24		24	100.00
Dummy	(3.0, 7.0)	-100	60.00	24		24	100.00
Dummy	(2.0, 5.0)	-100	60.00	24		24	100.00
Dummy	(3.0, 5.0)	-100	60.00	24		24	100.00
Dummy	(2.0, 2.0)	-10	60.00	24		24	100.00
Dummy	(2.0, 8.0)	-10	60.00	16		24	66.67
Dummy	(3.0, 3.0)	-10	60.00	24		24	100.00
Dummy	(3.0, 7.0)	-10	60.00	24		24	100.00
Dummy	(2.0, 5.0)	-10	60.00	24		24	100.00
Dummy	(3.0, 5.0)	-10	60.00	24		24	100.00
Dummy	(2.0, 2.0)	-1	60.00	24		24	100.00
Dummy	(2.0, 8.0)	-1	60.00	16		24	66.67
Dummy	(3.0, 3.0)	-1	60.00	24		24	100.00
Dummy	(3.0, 7.0)	-1	60.00	24		24	100.00
Dummy	(2.0, 5.0)	-1	60.00	24		24	100.00
Dummy	(3.0, 5.0)	-1	60.00	24		24	100.00
Trident	(6.75, 12.0)	-100	60.01	62		67	92.54
Trident	(8.5, 3.0)	-100	60.00	66		67	98.51
Trident	(10.5, 15.0)	-100	60.00	62		67	92.54
Trident	(13.5, 3.0)	-100	80.78	65		67	97.01
Trident	(10.5, 11.0)	-100	61.33	63		67	94.03
Trident	(15.5, 11.0)	-100	60.01	63		67	94.03
Trident	(10.5, 8.0)	-100	72.47	58		67	86.57
Trident	(15.5, 8.0)	-100	60.96	63		67	94.03
Trident	(6.75, 12.0)	-10	60.01	62		67	92.54
Trident	(8.5, 3.0)	-10	60.00	67		67	100.00
Trident	(10.5, 15.0)	-10	60.00	67		67	100.00
Trident	(13.5, 3.0)	-10	60.45	67		67	100.00
Trident	(10.5, 11.0)	-10	60.02	67		67	100.00
Trident	(15.5, 11.0)	-10	63.09	67		67	100.00
Trident	(10.5, 8.0)	-10	60.00	67		67	100.00
Trident	(15.5, 8.0)	-10	60.02	67		67	100.00
Trident	(6.75, 12.0)	-1	70.07	37		67	55.22
Trident	(8.5, 3.0)	-1	83.79	40		67	59.70
Trident	(10.5, 15.0)	-1	90.67	32		67	47.76
Trident	(13.5, 3.0)	-1	86.07	51		67	76.12
Trident	(10.5, 11.0)	-1	82.73	36		67	53.73
Trident	(15.5, 11.0)	-1	76.39	52		67	77.61
Trident	(10.5, 8.0)	-1	62.24	44		67	65.67
Trident	(15.5, 8.0)	-1	84.19	45		67	67.16
MGS	(18.6, 10.3)	-100	60.00	35		36	97.22
MGS	(13.6, 10.6)	-100	63.32	31		36	86.11
MGS	(1.5, 9.5)	-100	60.10	36		36	100.00
MGS	(7.0, 8.5)	-100	93.44	36		36	100.00
MGS	(9.5, 8.5)	-100	63.50	36		36	100.00
MGS	(13.0, 7.0)	-100	66.09	32		36	88.89
MGS	(6.5, 5.0)	-100	64.32	36		36	100.00
MGS	(4.0, 2.0)	-100	127.98	36		36	100.00
MGS	(8.0, 2.0)	-100	133.62	36		36	100.00
MGS	(13.0, 2.5)	-100	86.76	36		36	100.00
MGS	(18.6, 10.3)	-10	60.07	35		36	97.22
MGS	(13.6, 10.6)	-10	82.44	31		36	86.11
MGS	(1.5, 9.5)	-10	78.22	28		36	77.78
MGS	(7.0, 8.5)	-10	60.08	36		36	100.00
MGS	(9.5, 8.5)	-10	60.10	36		36	100.00
MGS	(13.0, 7.0)	-10	69.73	31		36	86.11
MGS	(6.5, 5.0)	-10	79.36	36		36	100.00
MGS	(4.0, 2.0)	-10	76.12	36		36	100.00
MGS	(8.0, 2.0)	-10	62.64	36		36	100.00

Continued on next page

Level	Start Position	Visibility Penalty (p)	Runtime (s)	Timesteps Hidden	Total Timesteps	Time Hidden (%)
MGS	(13.0, 2.5)	-10	60.03	36	36	100.00
MGS	(18.6, 10.3)	-1	159.33	25	36	69.44
MGS	(13.6, 10.6)	-1	225.91	19	36	52.78
MGS	(1.5, 9.5)	-1	60.00	23	36	63.89
MGS	(7.0, 8.5)	-1	60.23	22	36	61.11
MGS	(9.5, 8.5)	-1	114.72	23	36	63.89
MGS	(13.0, 7.0)	-1	221.39	18	36	50.00
MGS	(6.5, 5.0)	-1	98.14	28	36	77.78
MGS	(4.0, 2.0)	-1	129.34	14	36	38.89
MGS	(8.0, 2.0)	-1	113.22	30	36	83.33
MGS	(13.0, 2.5)	-1	85.73	22	36	61.11

Table 6: Full per-run results for Experiment 2, varying kernel distance weight (γ) and number of kernels.

Level	Start Position	Kernel Weight	# Kernels	Runtime (s)	Timesteps Hidden	Total Timesteps	Time Hidden (%)
Dummy	(2.0, 2.0)	1	1	60.00	24	24	100.00
Dummy	(2.0, 8.0)	1	1	60.00	16	24	66.67
Dummy	(3.0, 3.0)	1	1	60.00	24	24	100.00
Dummy	(3.0, 7.0)	1	1	60.00	24	24	100.00
Dummy	(2.0, 5.0)	1	1	60.00	24	24	100.00
Dummy	(3.0, 5.0)	1	1	60.00	24	24	100.00
Dummy	(2.0, 2.0)	1	3	60.00	24	24	100.00
Dummy	(2.0, 8.0)	1	3	60.00	16	24	66.67
Dummy	(3.0, 3.0)	1	3	60.00	24	24	100.00
Dummy	(3.0, 7.0)	1	3	60.00	24	24	100.00
Dummy	(2.0, 5.0)	1	3	60.00	24	24	100.00
Dummy	(3.0, 5.0)	1	3	60.00	24	24	100.00
Dummy	(2.0, 2.0)	2	1	60.00	24	24	100.00
Dummy	(2.0, 8.0)	2	1	60.00	16	24	66.67
Dummy	(3.0, 3.0)	2	1	60.00	24	24	100.00
Dummy	(3.0, 7.0)	2	1	60.00	24	24	100.00
Dummy	(2.0, 5.0)	2	1	60.00	24	24	100.00
Dummy	(3.0, 5.0)	2	1	60.00	24	24	100.00
Dummy	(2.0, 2.0)	4	1	60.00	24	24	100.00
Dummy	(2.0, 8.0)	4	1	60.00	16	24	66.67
Dummy	(3.0, 3.0)	4	1	60.00	24	24	100.00
Dummy	(3.0, 7.0)	4	1	60.00	24	24	100.00
Dummy	(2.0, 5.0)	4	1	60.00	24	24	100.00
Dummy	(3.0, 5.0)	4	1	60.00	24	24	100.00
Trident	(6.75, 12.0)	1	1	60.00	62	67	92.54
Trident	(8.5, 3.0)	1	1	60.00	67	67	100.00
Trident	(10.5, 15.0)	1	1	60.00	67	67	100.00
Trident	(13.5, 3.0)	1	1	60.45	67	67	100.00
Trident	(10.5, 11.0)	1	1	60.02	67	67	100.00
Trident	(15.5, 11.0)	1	1	63.09	67	67	100.00
Trident	(10.5, 8.0)	1	1	60.00	67	67	100.00
Trident	(15.5, 8.0)	1	1	60.02	67	67	100.00
Trident	(6.75, 12.0)	1	3	60.00	62	67	92.54
Trident	(8.5, 3.0)	1	3	60.00	66	67	98.51
Trident	(10.5, 15.0)	1	3	60.01	63	67	94.03
Trident	(13.5, 3.0)	1	3	60.02	67	67	100.00
Trident	(10.5, 11.0)	1	3	60.01	67	67	100.00
Trident	(15.5, 11.0)	1	3	60.01	67	67	100.00
Trident	(10.5, 8.0)	1	3	60.01	67	67	100.00
Trident	(15.5, 8.0)	1	3	68.17	67	67	100.00
Trident	(6.75, 12.0)	1	5	60.03	59	67	88.06
Trident	(8.5, 3.0)	1	5	60.00	65	67	97.01
Trident	(10.5, 15.0)	1	5	69.20	67	67	100.00
Trident	(13.5, 3.0)	1	5	60.01	67	67	100.00
Trident	(10.5, 11.0)	1	5	65.36	67	67	100.00
Trident	(15.5, 11.0)	1	5	60.01	67	67	100.00
Trident	(10.5, 8.0)	1	5	63.40	63	67	94.03
Trident	(15.5, 8.0)	1	5	60.00	62	67	92.54
Trident	(6.75, 12.0)	2	1	60.00	62	67	92.54
Trident	(8.5, 3.0)	2	1	60.00	66	67	98.51
Trident	(10.5, 15.0)	2	1	60.00	66	67	98.51
Trident	(13.5, 3.0)	2	1	60.00	67	67	100.00
Trident	(10.5, 11.0)	2	1	60.02	66	67	98.51
Trident	(15.5, 11.0)	2	1	60.00	67	67	100.00
Trident	(10.5, 8.0)	2	1	60.00	66	67	98.51
Trident	(15.5, 8.0)	2	1	60.00	67	67	100.00
Trident	(6.75, 12.0)	4	1	60.00	61	67	91.04
Trident	(8.5, 3.0)	4	1	60.00	66	67	98.51
Trident	(10.5, 15.0)	4	1	60.00	67	67	100.00
Trident	(13.5, 3.0)	4	1	60.00	67	67	100.00
Trident	(10.5, 11.0)	4	1	60.00	67	67	100.00

Continued on next page

Level	Start Position	Kernel Weight	# Kernels	Runtime (s)	Timesteps Hidden	Total Timesteps	Time Hidden (%)
Trident	(15.5, 11.0)	4	1	60.00	67	67	100.00
Trident	(10.5, 8.0)	4	1	60.00	64	67	95.52
Trident	(15.5, 8.0)	4	1	60.00	67	67	100.00
MGS	(18.6, 10.3)	1	1	60.11	33	36	91.67
MGS	(13.6, 10.6)	1	1	70.95	31	36	86.11
MGS	(1.5, 9.5)	1	1	63.37	35	36	97.22
MGS	(7.0, 8.5)	1	1	132.34	36	36	100.00
MGS	(9.5, 8.5)	1	1	163.42	36	36	100.00
MGS	(13.0, 7.0)	1	1	97.49	31	36	86.11
MGS	(6.5, 5.0)	1	1	168.43	36	36	100.00
MGS	(4.0, 2.0)	1	1	148.04	36	36	100.00
MGS	(8.0, 2.0)	1	1	78.90	36	36	100.00
MGS	(13.0, 2.5)	1	1	113.58	36	36	100.00
MGS	(18.6, 10.3)	1	3	60.02	35	36	97.22
MGS	(13.6, 10.6)	1	3	101.18	31	36	86.11
MGS	(1.5, 9.5)	1	3	60.29	32	36	88.89
MGS	(7.0, 8.5)	1	3	132.91	36	36	100.00
MGS	(9.5, 8.5)	1	3	95.23	36	36	100.00
MGS	(13.0, 7.0)	1	3	122.48	33	36	91.67
MGS	(6.5, 5.0)	1	3	239.40	36	36	100.00
MGS	(4.0, 2.0)	1	3	118.00	36	36	100.00
MGS	(8.0, 2.0)	1	3	114.50	36	36	100.00
MGS	(13.0, 2.5)	1	3	96.41	34	36	94.44
MGS	(18.6, 10.3)	1	5	60.15	34	36	94.44
MGS	(13.6, 10.6)	1	5	70.97	31	36	86.11
MGS	(1.5, 9.5)	1	5	64.63	36	36	100.00
MGS	(7.0, 8.5)	1	5	156.09	36	36	100.00
MGS	(9.5, 8.5)	1	5	116.71	36	36	100.00
MGS	(13.0, 7.0)	1	5	278.08	32	36	88.89
MGS	(6.5, 5.0)	1	5	151.29	36	36	100.00
MGS	(4.0, 2.0)	1	5	155.88	36	36	100.00
MGS	(8.0, 2.0)	1	5	183.12	36	36	100.00
MGS	(13.0, 2.5)	1	5	93.85	36	36	100.00
MGS	(18.6, 10.3)	2	5	60.06	35	36	97.22
MGS	(13.6, 10.6)	2	5	62.17	31	36	86.11
MGS	(1.5, 9.5)	2	5	63.90	36	36	100.00
MGS	(7.0, 8.5)	2	5	212.29	36	36	100.00
MGS	(9.5, 8.5)	2	5	88.83	34	36	94.44
MGS	(13.0, 7.0)	2	5	81.77	33	36	91.67
MGS	(6.5, 5.0)	2	5	336.41	36	36	100.00
MGS	(4.0, 2.0)	2	5	102.99	36	36	100.00
MGS	(8.0, 2.0)	2	5	129.33	36	36	100.00
MGS	(13.0, 2.5)	2	5	116.33	36	36	100.00
MGS	(18.6, 10.3)	4	5	60.06	35	36	97.22
MGS	(13.6, 10.6)	4	5	83.33	29	36	80.56
MGS	(1.5, 9.5)	4	5	96.76	36	36	100.00
MGS	(7.0, 8.5)	4	5	210.69	35	36	97.22
MGS	(9.5, 8.5)	4	5	177.34	36	36	100.00
MGS	(13.0, 7.0)	4	5	104.50	31	36	86.11
MGS	(6.5, 5.0)	4	5	141.92	36	36	100.00
MGS	(4.0, 2.0)	4	5	99.59	36	36	100.00
MGS	(8.0, 2.0)	4	5	137.50	36	36	100.00
MGS	(13.0, 2.5)	4	5	80.57	36	36	100.00

Table 7: Full per-run results for Experiment 3, varying rollout weight (λ) and depth.

Level	Start Position	Rollout Weight	Rollout Depth	Runtime (s)	Timesteps Hidden	Total Timesteps	Time Hidden (%)
Dummy	(2.0, 2.0)	1	7	60.00	24	24	100.00
Dummy	(2.0, 8.0)	1	7	60.00	17	24	70.83
Dummy	(3.0, 3.0)	1	7	60.00	24	24	100.00
Dummy	(3.0, 7.0)	1	7	60.00	24	24	100.00
Dummy	(2.0, 5.0)	1	7	60.00	24	24	100.00
Dummy	(3.0, 5.0)	1	7	60.00	24	24	100.00
Dummy	(2.0, 2.0)	2	7	60.00	24	24	100.00
Dummy	(2.0, 8.0)	2	7	60.00	17	24	70.83
Dummy	(3.0, 3.0)	2	7	60.00	24	24	100.00
Dummy	(3.0, 7.0)	2	7	60.00	24	24	100.00
Dummy	(2.0, 5.0)	2	7	60.00	24	24	100.00
Dummy	(3.0, 5.0)	2	7	60.00	24	24	100.00
Dummy	(2.0, 2.0)	2	15	60.00	24	24	100.00
Dummy	(2.0, 8.0)	2	15	60.00	17	24	70.83
Dummy	(3.0, 3.0)	2	15	60.00	24	24	100.00
Dummy	(3.0, 7.0)	2	15	60.00	24	24	100.00
Dummy	(2.0, 5.0)	2	15	60.00	24	24	100.00
Dummy	(3.0, 5.0)	2	15	60.00	24	24	100.00
Dummy	(2.0, 2.0)	2	30	60.00	24	24	100.00

Continued on next page

Level	Start Position	Rollout Weight	Rollout Depth	Runtime (s)	Timesteps Hidden	Total Timesteps	Time Hidden (%)
Dummy	(2.0, 8.0)	2	30	60.00	17	24	70.83
Dummy	(3.0, 3.0)	2	30	60.00	24	24	100.00
Dummy	(3.0, 7.0)	2	30	60.00	24	24	100.00
Dummy	(2.0, 5.0)	2	30	60.00	24	24	100.00
Dummy	(3.0, 5.0)	2	30	60.00	24	24	100.00
Dummy	(2.0, 2.0)	4	7	60.00	24	24	100.00
Dummy	(2.0, 8.0)	4	7	60.00	17	24	70.83
Dummy	(3.0, 3.0)	4	7	60.00	24	24	100.00
Dummy	(3.0, 7.0)	4	7	60.00	24	24	100.00
Dummy	(2.0, 5.0)	4	7	60.00	24	24	100.00
Dummy	(3.0, 5.0)	4	7	60.00	24	24	100.00
Trident	(6.75, 12.0)	1	30	85.97	62	67	92.54
Trident	(8.5, 3.0)	1	30	69.01	66	67	98.51
Trident	(10.5, 15.0)	1	30	65.01	62	67	92.54
Trident	(13.5, 3.0)	1	30	60.84	60	67	89.55
Trident	(10.5, 11.0)	1	30	71.00	65	67	97.01
Trident	(15.5, 11.0)	1	30	60.02	63	67	94.03
Trident	(10.5, 8.0)	1	30	73.46	65	67	97.01
Trident	(15.5, 8.0)	1	30	60.00	63	67	94.03
Trident	(6.75, 12.0)	2	30	61.95	63	67	94.03
Trident	(8.5, 3.0)	2	30	62.33	61	67	91.04
Trident	(10.5, 15.0)	2	30	69.83	63	67	94.03
Trident	(13.5, 3.0)	2	30	61.90	62	67	92.54
Trident	(10.5, 11.0)	2	30	64.14	65	67	97.01
Trident	(15.5, 11.0)	2	30	60.00	63	67	94.03
Trident	(10.5, 8.0)	2	30	60.00	63	67	94.03
Trident	(15.5, 8.0)	2	30	60.00	60	67	89.55
Trident	(6.75, 12.0)	4	7	60.01	64	67	95.52
Trident	(8.5, 3.0)	4	7	60.01	66	67	98.51
Trident	(10.5, 15.0)	4	7	61.45	64	67	95.52
Trident	(13.5, 3.0)	4	7	60.01	63	67	94.03
Trident	(10.5, 11.0)	4	7	65.42	62	67	92.54
Trident	(15.5, 11.0)	4	7	60.01	64	67	95.52
Trident	(10.5, 8.0)	4	7	60.01	64	67	95.52
Trident	(15.5, 8.0)	4	7	60.01	64	67	95.52
Trident	(6.75, 12.0)	4	15	60.00	64	67	95.52
Trident	(8.5, 3.0)	4	15	60.68	64	67	95.52
Trident	(10.5, 15.0)	4	15	60.56	64	67	95.52
Trident	(13.5, 3.0)	4	15	60.00	62	67	92.54
Trident	(10.5, 11.0)	4	15	60.01	63	67	94.03
Trident	(15.5, 11.0)	4	15	60.01	65	67	97.01
Trident	(10.5, 8.0)	4	15	60.01	63	67	94.03
Trident	(15.5, 8.0)	4	15	60.01	63	67	94.03
Trident	(6.75, 12.0)	4	30	60.01	62	67	92.54
Trident	(8.5, 3.0)	4	30	60.00	67	67	100.00
Trident	(10.5, 15.0)	4	30	60.45	67	67	100.00
Trident	(13.5, 3.0)	4	30	60.02	67	67	100.00
Trident	(10.5, 11.0)	4	30	63.09	67	67	100.00
Trident	(15.5, 11.0)	4	30	60.00	67	67	100.00
Trident	(10.5, 8.0)	4	30	60.02	67	67	100.00
Trident	(15.5, 8.0)	4	30	60.02	67	67	100.00
MGS	(18.6, 10.3)	1	7	60.01	35	36	97.22
MGS	(13.6, 10.6)	1	7	60.23	30	36	83.33
MGS	(1.5, 9.5)	1	7	60.12	35	36	97.22
MGS	(7.0, 8.5)	1	7	168.45	36	36	100.00
MGS	(9.5, 8.5)	1	7	90.83	36	36	100.00
MGS	(13.0, 7.0)	1	7	104.60	28	36	77.78
MGS	(6.5, 5.0)	1	7	202.13	36	36	100.00
MGS	(4.0, 2.0)	1	7	79.45	36	36	100.00
MGS	(8.0, 2.0)	1	7	133.78	36	36	100.00
MGS	(13.0, 2.5)	1	7	76.10	36	36	100.00
MGS	(18.6, 10.3)	2	7	60.00	35	36	97.22
MGS	(13.6, 10.6)	2	7	63.32	31	36	86.11
MGS	(1.5, 9.5)	2	7	60.10	36	36	100.00
MGS	(7.0, 8.5)	2	7	93.44	36	36	100.00
MGS	(9.5, 8.5)	2	7	63.50	36	36	100.00
MGS	(13.0, 7.0)	2	7	66.09	32	36	88.89
MGS	(6.5, 5.0)	2	7	64.32	36	36	100.00
MGS	(4.0, 2.0)	2	7	127.98	36	36	100.00
MGS	(8.0, 2.0)	2	7	133.62	36	36	100.00
MGS	(13.0, 2.5)	2	7	86.76	36	36	100.00
MGS	(18.6, 10.3)	2	15	60.04	35	36	97.22
MGS	(13.6, 10.6)	2	15	70.42	35	36	97.22
MGS	(1.5, 9.5)	2	15	158.67	35	36	97.22
MGS	(7.0, 8.5)	2	15	65.68	36	36	100.00
MGS	(9.5, 8.5)	2	15	107.66	29	36	80.56
MGS	(13.0, 7.0)	2	15	153.51	36	36	100.00
MGS	(6.5, 5.0)	2	15	107.94	36	36	100.00
MGS	(4.0, 2.0)	2	15	75.79	34	36	94.44

Continued on next page

Level	Start Position	Rollout Weight	Rollout Depth	Runtime (s)	Timesteps Hidden	Total Timesteps	Time Hidden (%)
MGS	(8.0, 2.0)	2	15	71.52	36	36	100.00
MGS	(13.0, 2.5)	2	15	71.52	36	36	100.00
MGS	(18.6, 10.3)	2	30	60.05	30	36	83.33
MGS	(13.6, 10.6)	2	30	73.35	30	36	83.33
MGS	(1.5, 9.5)	2	30	60.08	36	36	100.00
MGS	(7.0, 8.5)	2	30	112.39	36	36	100.00
MGS	(9.5, 8.5)	2	30	106.33	36	36	100.00
MGS	(13.0, 7.0)	2	30	63.27	33	36	91.67
MGS	(6.5, 5.0)	2	30	90.83	36	36	100.00
MGS	(4.0, 2.0)	2	30	84.08	36	36	100.00
MGS	(8.0, 2.0)	2	30	79.80	35	36	97.22
MGS	(13.0, 2.5)	2	30	72.06	36	36	100.00
MGS	(18.6, 10.3)	4	7	60.02	35	36	97.22
MGS	(13.6, 10.6)	4	7	60.04	34	36	94.44
MGS	(1.5, 9.5)	4	7	60.18	36	36	100.00
MGS	(7.0, 8.5)	4	7	84.54	36	36	100.00
MGS	(9.5, 8.5)	4	7	60.06	36	36	100.00
MGS	(13.0, 7.0)	4	7	60.05	35	36	97.22
MGS	(6.5, 5.0)	4	7	104.75	36	36	100.00
MGS	(4.0, 2.0)	4	7	67.32	36	36	100.00
MGS	(8.0, 2.0)	4	7	79.69	36	36	100.00
MGS	(13.0, 2.5)	4	7	68.83	34	36	94.44

Table 8: Full per-run results for Experiment 4, adding brute-force actions.

Level	Start Position	Brute-Force Actions	Runtime (s)	Timesteps Hidden	Total Timesteps	Time Hidden (%)
Dummy	(2.0, 2.0)	TRUE	60.00	24	24	100.00
Dummy	(2.0, 8.0)	TRUE	60.00	14	24	58.33
Dummy	(3.0, 3.0)	TRUE	60.00	24	24	100.00
Dummy	(3.0, 7.0)	TRUE	60.00	24	24	100.00
Dummy	(2.0, 5.0)	TRUE	60.00	24	24	100.00
Dummy	(3.0, 5.0)	TRUE	60.00	24	24	100.00
Dummy	(2.0, 2.0)	FALSE	60.00	24	24	100.00
Dummy	(2.0, 8.0)	FALSE	60.00	17	24	70.83
Dummy	(3.0, 3.0)	FALSE	60.00	24	24	100.00
Dummy	(3.0, 7.0)	FALSE	60.00	24	24	100.00
Dummy	(2.0, 5.0)	FALSE	60.00	24	24	100.00
Dummy	(3.0, 5.0)	FALSE	60.00	24	24	100.00
Trident	(6.75, 12.0)	TRUE	84.73	63	67	94.03
Trident	(8.5, 3.0)	TRUE	60.02	66	67	98.51
Trident	(10.5, 15.0)	TRUE	60.00	67	67	100.00
Trident	(13.5, 3.0)	TRUE	94.97	62	67	92.54
Trident	(10.5, 11.0)	TRUE	151.64	66	67	98.51
Trident	(15.5, 11.0)	TRUE	107.31	62	67	92.54
Trident	(10.5, 8.0)	TRUE	94.21	66	67	98.51
Trident	(15.5, 8.0)	TRUE	115.31	67	67	100.00
Trident	(6.75, 12.0)	FALSE	60.01	62	67	92.54
Trident	(8.5, 3.0)	FALSE	60.00	67	67	100.00
Trident	(10.5, 15.0)	FALSE	60.00	67	67	100.00
Trident	(13.5, 3.0)	FALSE	60.45	67	67	100.00
Trident	(10.5, 11.0)	FALSE	60.02	67	67	100.00
Trident	(15.5, 11.0)	FALSE	63.09	67	67	100.00
Trident	(10.5, 8.0)	FALSE	60.00	67	67	100.00
Trident	(15.5, 8.0)	FALSE	60.02	67	67	100.00
MGS	(18.6, 10.3)	TRUE	60.13	35	36	97.22
MGS	(13.6, 10.6)	TRUE	73.42	31	36	86.11
MGS	(1.5, 9.5)	TRUE	60.01	33	36	91.67
MGS	(7.0, 8.5)	TRUE	93.47	36	36	100.00
MGS	(9.5, 8.5)	TRUE	116.20	36	36	100.00
MGS	(13.0, 7.0)	TRUE	95.64	31	36	86.11
MGS	(6.5, 5.0)	TRUE	152.42	36	36	100.00
MGS	(4.0, 2.0)	TRUE	120.04	36	36	100.00
MGS	(8.0, 2.0)	TRUE	119.83	36	36	100.00
MGS	(13.0, 2.5)	TRUE	98.06	34	36	94.44
MGS	(18.6, 10.3)	FALSE	60.00	35	36	97.22
MGS	(13.6, 10.6)	FALSE	63.32	31	36	86.11
MGS	(1.5, 9.5)	FALSE	60.10	36	36	100.00
MGS	(7.0, 8.5)	FALSE	93.44	36	36	100.00
MGS	(9.5, 8.5)	FALSE	63.50	36	36	100.00
MGS	(13.0, 7.0)	FALSE	66.09	32	36	88.89
MGS	(6.5, 5.0)	FALSE	64.32	36	36	100.00
MGS	(4.0, 2.0)	FALSE	127.98	36	36	100.00
MGS	(8.0, 2.0)	FALSE	133.62	36	36	100.00
MGS	(13.0, 2.5)	FALSE	86.76	36	36	100.00